Federate the Decision, Not the Model: Learning Tool-Routing Controllers for Inspection Drone Fleets under One-Sided Feedback

Mahdi Khashan

[a][Affiliation to be added],

Abstract

An inspection drone has to decide for every frame whether to trust its onboard classifier, run a slower onboard vision-language model (VLM), or send the frame to a cloud VLM over a weak link. A small learned controller can make this decision far better than a fixed confidence cascade, but one drone collects too few informative frames to train it, and the obvious way to pool a fleet's knowledge, federated fine-tuning of the perception model, is expensive: across 42 federated configurations of a 201M-parameter vision-language decision model on power-line images, the fleet sends 425 to 41,310 MB and small updates double the calibration error. We instead federate the controller. Operators in five regions, each holding its own towers and labels, train a 29 KB multilayer perceptron with FedAvg in a replay of 25,499 frames with LTE uplink traces, a 4 s deadline and an onboard queue. With full labels, the federated controller reaches a mission risk of 115.4 ± 1.6 per 1,000 test frames (3 seeds), equal to central training (114.6) and well below the static cascade (155.7) and regions training alone (157.9 ± 20.4), with about 7 MB of total uplink. In deployment the feedback is one-sided: accepting an onboard answer reveals nothing, the cloud VLM is a noisy teacher (accuracy 0.79), and operator labels arrive only for frames that are sent and reviewed. Under this feedback, learning from cloud answers makes the controller worse than the cascade (175.7–188.4), and only federated learning from operator labels with a 5% audit of accepted frames beats it (142.4 ± 3.6), at about 76 MB of audit and escalation uploads. Regions alone with the same audit do not (157.3). The decision layer, not the perception model, is where a fleet gains most per transmitted byte; what limits it is the feedback.

Keywords: drone fleets, federated learning, edge AI, tool routing, partial feedback, vision-language model, infrastructure inspection

1. Introduction

Infrastructure inspection is moving from drones that record video for later analysis to fleets that interpret what they see on board and act on it [1]. On each frame, a drone chooses among perception tools of different cost: a fast onboard classifier, a slower onboard VLM, or a larger VLM in the cloud reached over a link that is often weak or down. Farahani et al. run a fine-tuned SmolVLM-256M on a Jetson in 1.4–6.4 s per segment [2], and profile whether such a model should run on board, at the edge or split [3]; both name an adaptive runtime orchestrator as future work.

In our replay of a power-line mission, the choice of tool matters more than any single tool. A static cascade that escalates low-confidence frames to the onboard VLM has a risk of 155.7 per 1,000 frames; an oracle that knows which tool is right needs only 62.9 and escalates 3.1% of frames. A small learned controller closes part of this gap, but it needs about 1,300–5,100 labelled frames before it beats the cascade, more than one drone or one operator collects quickly. Pooling across the fleet is therefore attractive.

The common way to pool a fleet's experience is to federate the perception model [11]. We did this for laya-vision [6], a 201M-parameter vision-language decision model, over 42 configurations. It works, but it costs 425–41,310 MB of uplink per training run, and the cheaper updates lift defect ranking while doubling the calibration error the controller relies on. This paper therefore asks a different question: what does a fleet gain if it federates the decision instead, a controller of 29 KB?

Two properties of deployment make this non-trivial. First, operators keep their images and labels; we model each client as an operator region (cross-silo), so central training is not available. Second, the controller's own choices decide which of its mistakes it ever sees. Accepting an onboard answer reveals nothing; escalating to the cloud returns an answer from a model that is right only 79% of the time; operator labels arrive only for frames that are sent and reviewed. Zhang et al. [8] study this policy-dependent feedback for hierarchical inference with a ground-truth oracle at the top. Here the top layer is noisy, labels are selective, and the learners are federated.

We contribute (i) a replay benchmark for federated tool routing on 25,499 training and 7,057 test frames with LTE

Email address: mahdikhashan1@gmail.com (Mahdi Khashan)

traces, a deadline and an onboard queue; (ii) evidence that a federated 29 KB controller matches central training and beats the static cascade by 26% at about 7 MB of uplink, while regions alone do not; (iii) a feedback model with unobserved accepts, noisy cloud answers and audited operator labels, under which we compare five learners locally and federated; and (iv) a comparison with federating the perception model, which costs two to four orders of magnitude more uplink.

2. Related work

Edge VLMs for inspection drones. EVLM [2] runs a LoRA-tuned SmolVLM-256M on a Jetson and cuts the uplink of a 4 s segment from 485 kB to 25 kB; [3] profiles onboard, edge and split deployment, and INAR-VL [7] routes requests between edge and cloud with a bandwidth guard. All use fixed rules. Laya [5] and laya-vision [6] answer typed questions in one forward pass with calibrated probabilities, which we use as the federated perception model in Section 5.

Routing, cascades and partial feedback. Federate the Router [14] learns language-model routers with FedAvg from offline evaluations and leaves online routing as future work; FedHLM [15] federates escalation thresholds. BaRP [9] trains a router from bandit feedback, where only the chosen model's outcome is observed. Zhang et al. [8] show that importance-weighted bandit estimators become unstable when the probability of reaching the ground-truth oracle decays along the hierarchy, and add uniform exploration and variance reduction. Agarwal et al. [10] evaluate federated contextual bandits empirically. We combine these settings: federated clients, a noisy upper tool, and selective, audited ground truth.

Federated learning. FedAvg [11] averages client models weighted by data size; non-IID data degrades it [12], and FedProx [13] adds a proximal term.

3. System

3.1. Tools, mission and cost

We use the tower14 split of ElecVQA-Bench [16], built on InsPLAD power-line component crops [17]: 18 training towers (25,499 items), 10 validation towers (6,672) and 14 test towers (7,057). Each item asks whether a component has a defect, or which condition it has. Every tool was run once on every item, and the replay reads these cached answers:

- classifier: a ResNet-50 [19] fine-tuned on the training towers, run on every frame (test accuracy 0.961);
- local: SmolVLM-256M [4] with LoRA [20] as in EVLM, on board (0.956);
- cloud: Qwen3-VL-8B zero-shot (0.792; corrosion recall 0.012), which needs a 35 kB JPEG upload.

Table 1: Regions. Informative frames: frames on which the three actions differ in risk by more than the escalation cost.

Region	Towers	Frames	Defects	Informative
R1	1	6,449	490	1,011
R2	1	5,297	639	935
R3	5	4,581	619	1,017
R4	5	4,579	599	989
R5	6	4,593	775	1,074

The tools make different errors: of the classifier's 279 test errors, the onboard VLM fixes 186 and the cloud 135.

A mission replays the items in tower order with one frame every 1.16 s [2] and a deadline of 4 s; a late answer counts as a miss. Onboard tools share one queue; their times are GV100 times multiplied by 9, our estimate of the gap to a Jetson Orin NX. The uplink follows 4G/LTE traces [18] scaled to a weak regime (10% of the trace bandwidth, six 20 s outages per hour); a cloud request fails when its transfer takes more than 3 s. Mission risk per 1,000 frames counts 10 per missed defect, 1 per false alarm and 0.5 per abstention. Results are means over 5 trace seeds on the test towers.

3.2. The controller

After the classifier, the controller chooses one action: accept the classifier's answer, ask the local VLM, or send the frame to the cloud. Its input is the task type, the component type, the classifier's confidence, top-two margin, entropy, predicted label and class probabilities, and five image statistics (size, blur, brightness, contrast). A multilayer perceptron with two hidden layers of 64 units (7,171 parameters, 28.7 KB in fp32) outputs a probability per action. It is trained with soft cross-entropy against $\mathrm{softmax}(-r_a)$, where r_a is the risk of action a on the frame plus a small escalation cost (0.1 local, 0.2 cloud); frames on which the actions differ are oversampled five times. A fixed rule turns the output into a decision: escalate to the best tool whose probability exceeds that of accepting by a margin, and only if the tool can meet the deadline under the current link and queue. The margin is tuned on the validation towers. The network state thus enters through the rule, which stays on the drone, and not through the federated weights.

3.3. Regions and federated training

A client is an operator region that keeps its images and labels. We split the 18 training towers into five regions of balanced size (Table 1). Two regions hold a single large tower, so the split is strongly non-IID. The server holds the validation towers, which it uses only to stop training and to tune the margin.

Feature normalization uses federated means and variances. In each FedAvg round, every region runs 10 full-batch Adam steps (learning rate 10^{-3}) from the global

Table 2: What each action reveals to the learner.

Action	Feedback
Accept	Nothing; with probability λ an audit gives the operator label.
Local	The onboard VLM's answer, not a label; audit as for accept.
Cloud	The cloud answer (noisy); with probability ρ the operator label.

weights, and the server averages the weights by region size. Training stops 30 rounds after the best validation loss. FedProx adds a proximal term with $\mu = 0.01$. References are central training on all regions and each region alone.

3.4. One-sided feedback

Section 4.1 gives every learner the risk of all actions on every training frame. Deployment gives far less (Table 2). Each region processes its frames in tower order over 10 rounds. In round 0 it uses the static cascade; afterwards it uses the action with the highest probability under its current controller, replaced by a uniform random action with probability $\varepsilon = 0.05$. The probability of every action is logged. Accepted and locally answered frames are audited, that is, sent to the operator without changing the answer, with probability λ. Cloud frames are reviewed with probability $\rho = 0.5$. A reviewed frame reaches the ground station, which runs all tools on it, so its operator label reveals the risk of all three actions. After each round the controller trains on everything its region has revealed so far: 20 FedAvg rounds of 10 steps in the federated setting, or 200 steps for a region alone.

We compare five learners. Full labels receives the true risks on every frame and is the upper bound. Cloud as truth treats the cloud answer of unreviewed cloud frames as the label. True labels uses operator labels only. True labels, IPW weights each operator label by $1/P(\text{label})$ with $P(\text{label}) = \pi(\text{cloud})\,\rho + (1 - \pi(\text{cloud}))\,\lambda$, clipped at 0.02. Multi-fidelity adds unreviewed cloud frames with a soft target that averages over the true label given the cloud answer, estimated from frames that have both. The learner sees only this feedback; the evaluation uses every cached answer and the true labels of the test towers.

4. Results

4.1. Full feedback: federation matches central training

With labels for every frame, FedAvg reaches a risk of 115.4 ± 1.6, statistically indistinguishable from central training (114.6 ± 3.1) and 26% below the static cascade (Fig. 1a, Table 3). It finds more defects (recall 0.923 against 0.896) at a false-alarm rate of 0.023 and escalates 21% of frames. FedProx gives the same result. The best validation loss comes after 16–24 rounds; the runs stop after 47–55, so the total uplink of five regions is 6.7–7.9 MB, of which a third would suffice.

Regions alone fail in a way that matters for the argument. The two single-tower regions never learn to escalate: in all six of their runs the tuned margin turns escalation off, and the controller collapses to the classifier (177.0). The multi-tower regions reach 129–159. On average a region alone is no better than the cascade (157.9 ± 20.4), although every region holds about a thousand informative frames. What a region lacks is variety, not volume, and federation supplies it at 29 KB per update.

4.2. One-sided feedback

Online learning with full labels gives 119.7 ± 4.8, close to the offline result. With the feedback of deployment, the picture changes (Fig. 1b).

Cloud answers are harmful teachers. Treating them as labels yields 175.7–188.4 at every audit rate, worse than the cascade and close to the classifier alone. The cloud is wrong on 1,322 test items where the classifier is right, so its answers teach the controller to distrust correct onboard answers. While learning, it sends 2,500–3,600 frames to the cloud and spends 129–145 MB of uploads.

Operator labels are too few without audits. Without audits, the two true-label learners collect 372–386 operator labels across all five regions, from exploration and reviewed cloud frames, and end at 149.6 and 162.8, around the cascade. With a 5% audit they collect about 1,720 labels and reach 141.6 ± 11.5 (unweighted) and 142.4 ± 3.6 (IPW). IPW does not lower the mean, but cuts the spread across seeds by a factor of three. At 1–2% audit, no learner is clearly below the cascade. These label counts match the offline learning curve: a controller needs more than about 1,300 labels to beat the cascade.

Federation is necessary. With the same 5% audit, regions alone reach 157.3 ± 5.3 with IPW and 167.3 with the multi-fidelity learner; even with full labels, regions alone reach only 155.7. Every federated learner with operator labels beats its local counterpart.

The multi-fidelity learner does not help in its current form (156.8–160.7). Our reading, not tested, is that its estimate of the cloud's confusion rests on too few reviewed frames, so soft targets from a 79%-accurate teacher still pull the controller towards the cloud's errors.

Labels cost more uplink than models. A 5% audit uploads about 1,270 frames, and the run sends 76 MB of frames in total, more than twice the 28.7 MB of controller updates (1,000 updates of 28.7 KB, because online training synchronizes 20 times per round). In this fleet the learning budget is dominated by feedback, not by model synchronization.

5. Federating perception instead

The alternative is to federate the perception model and keep a fixed rule. We studied laya-vision [6] (thaitea/laya-vision, 201M parameters, SmolVLM-256M cut to 20 language layers plus a decision head), which answers component, defect and condition queries in one forward pass.

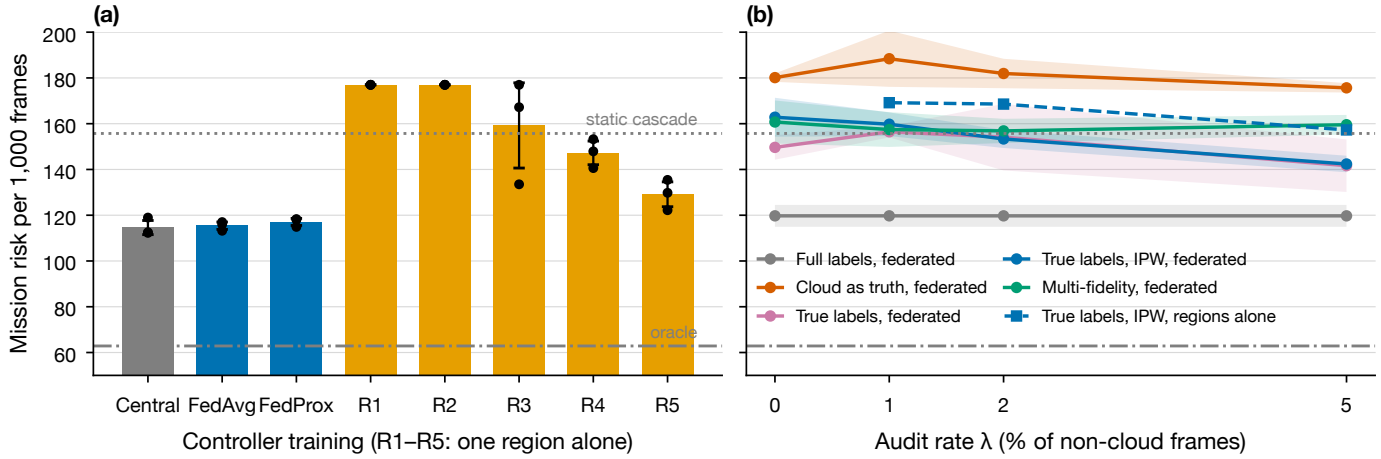


Figure 1: A federated 29 KB controller matches central training; under one-sided feedback only audited operator labels, pooled across regions, beat the cascade. Mission risk per 1,000 test frames, weak network. (a) Full feedback (offline): bars are means over 3 training seeds, dots the seeds, whiskers one standard deviation. R1–R5 train alone. (b) Online learning over 10 rounds against the audit rate λ; bands are one standard deviation over 3 seeds. The dotted line is the static cascade, the dash-dotted line the oracle.

Zero-shot it fails (condition macro F1 0.180). Fine-tuned centrally on folder labels of 4,807 training images, it reaches macro F1 0.703, defect AUROC 0.965 and 0.948 item accuracy at 74 ms per image on an RTX 2080 Ti, below the ResNet-50 classifier (0.961).

We then federated it in 42 configurations: 3 to 18 drones, the decision head, LoRA of rank 8, the last 4 or all 20 language layers, or the full model; fp16 or int8 updates; from the published checkpoint or from a server model fine-tuned on 13 server-held towers; few labels per drone; and drones that see different components. Each configuration has one seed. Figure 2 and Table 4 summarize them.

Three results bear on the controller. First, the cost: one full update is 115 MB in int8 (230 MB in fp16), a training run 0.4–41 GB, which is two to four orders of magnitude more than the 7 MB of a federated controller. From the published checkpoint, updates below 3 GB barely learn (macro F1 0.24–0.29). Second, from the server model every update lifts defect AUROC from 0.903 to 0.940–0.962, and int8 full updates match fp16 at half the uplink (macro F1 0.638 against 0.636). Third, the small updates (head, LoRA, last 4) double the calibration error of the defect probability (0.06 to 0.14–0.15), because each fleet drone's images hold 34–67% defects while the test towers hold 10%. A controller that thresholds probabilities would be misled. A Platt fit on the server's validation images repairs this (ECE 0.024–0.027), so perception updates must come with recalibration. With 50 labels per drone, FedAvg even lowers the AUROC below the server model (0.839), and only balanced local sampling avoids it (0.925).

6. Discussion

What a fleet should federate. The controller and the perception model improve different things, and they cost very different amounts. A federated controller gains 40 risk points over the cascade for about 7 MB; federating perception improves the defect ranking for hundreds of megabytes to gigabytes, and cheap perception updates must be recalibrated before a controller can use them. For a fleet with a weak uplink, our results suggest federating the decision first. Combining a federated controller with the federated perception model is the next experiment; it needs the layavision outputs on the training towers.

The limit is feedback. With full labels the federated controller is near central training; with deployment feedback it loses most of that gain. The controller sees its mistakes only where it escalates or where the operator audits, cloud answers are worse than no labels, and only a few percent of audited frames, pooled across regions, restore an advantage over the cascade. Audit and escalation uploads exceed the controller updates in the uplink of learning. Warm-starting the controller from labels the server already holds, variance-reduced estimators [8], and audits that target uncertain frames are the obvious ways to lower this cost.

Limitations. The fleet and the network are simulated: tool answers are cached, the Jetson latency is a factor-9 estimate, the uplink regime is constructed from LTE traces, and the risk weights are assumptions. The federated controller results use 3 training seeds and one network regime; the perception results use one seed. The controller does not see the network state; the deadline rule handles it. The data cover one benchmark and one asset type.

7. Conclusion

A 29 KB tool-routing controller, trained with FedAvg across operator regions that keep their images and labels, matches central training (risk 115.4 against 114.6 per 1,000

Table 3: Controllers on the test towers, weak network, mean ± standard deviation over 3 training seeds (5 trace seeds each). Risk counts 10 per missed defect, 1 per false alarm and 0.5 per abstention per 1,000 frames. Model uplink: total controller updates sent during training. Training uploads: frames uploaded by audits and cloud escalations while learning (online rows). Bold: best per group.

Setting	Controller	Risk	Defect recall	Escalation	True labels	Model uplink	Training uploads
No learning							
	Classifier only	177.0	0.888	0	–	–	–
	Static cascade	155.7	0.896	0.144	–	–	–
	Oracle (hindsight)	62.9	0.952	0.031	–	–	–
Full feedback, offline							
	Central	114.6 ± 3.1	0.923	0.186	25,499	images pooled	–
	FedAvg	115.4 ± 1.6	0.923	0.212	25,499	6.7–7.9 MB	–
	FedProx	117.1 ± 1.6	0.921	0.178	25,499	7.0–8.3 MB	–
	Regions alone (15 models)	157.9 ± 20.4	0.897	0.093	4,579–6,449	0	–
One-sided feedback, online, federated							
	Full labels	119.7 ± 4.8	0.920	0.063	25,499	28.7 MB	–
	Cloud as truth, $\lambda = 5\%$	175.7 ± 2.1	0.901	0.047	2,443	28.7 MB	141 MB
	True labels, $\lambda = 0$	149.6 ± 5.4	0.901	0.120	386	28.7 MB	30 MB
	True labels, $\lambda = 5\%$	141.6 ± 11.5	0.905	0.079	1,719	28.7 MB	75 MB
	True labels, IPW, $\lambda = 2\%$	153.3 ± 3.9	0.897	0.060	985	28.7 MB	52 MB
	True labels, IPW, $\lambda = 5\%$	142.4 ± 3.6	0.907	0.072	1,727	28.7 MB	76 MB
	Multi-fidelity, $\lambda = 2\%$	156.8 ± 5.3	0.897	0.069	1,052	28.7 MB	58 MB
One-sided feedback, online, regions alone (mean of 5 regions)							
	Full labels	155.7 ± 2.3	0.897	0.044	25,499	0	–
	True labels, IPW, $\lambda = 5\%$	157.3 ± 5.3	0.898	0.060	2,047	0	101 MB
	Multi-fidelity, $\lambda = 5\%$	167.3 ± 3.2	0.893	0.035	2,242	0	119 MB

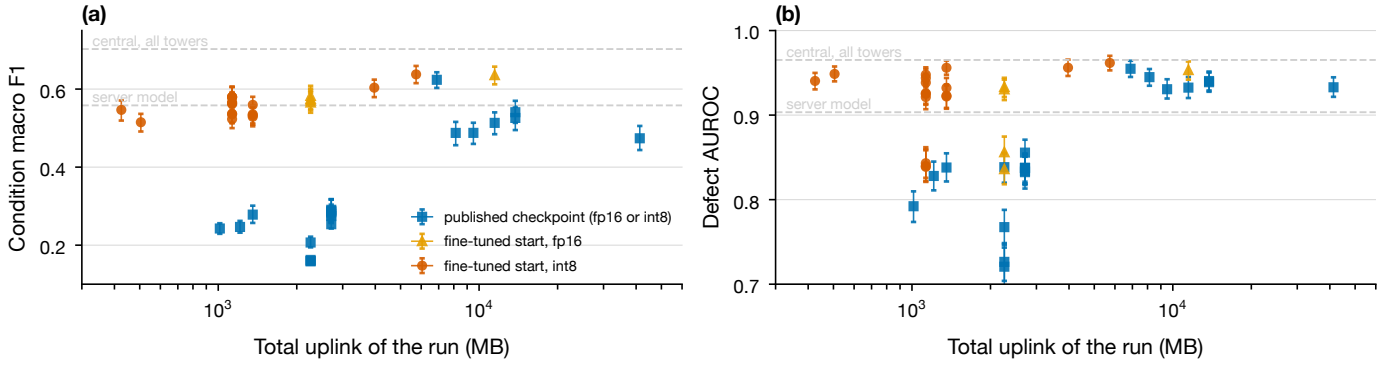


Figure 2: Federating the perception model costs gigabytes, and the start matters more than the uplink. Test condition macro F1 (a) and defect AUROC (b) of laya-vision for all 42 federated configurations against the total uplink of the run, by start. Whiskers are 95% bootstrap intervals; dashed lines mark central training and the server model. One seed per run.

Table 4: Federated laya-vision from the server model (5 fleet drones, 10 rounds, int8 updates; one seed). F1: condition macro F1. ECE of the defect probability on the test towers, raw and after a two-parameter Platt fit on 1,500 validation images (Platt).

Update	Uplink	F1	AUROC	ECE	Platt
Server model	0	0.558	0.903	0.063	0.027
Head	425 MB	0.547	0.940	0.149	0.027
LoRA r8	505 MB	0.515	0.949	0.143	0.024
Last 4 layers	1,130 MB	0.537	0.948	0.144	0.025
20 layers	3,965 MB	0.604	0.956	0.085	0.016
Full	5,735 MB	0.638	0.962	0.065	0.021

frames) and beats a static cascade by 26% for about 7 MB of uplink, whereas regions alone do not beat the cascade. Federating the perception model instead costs 0.4–41 GB and needs recalibration. In deployment, the controller's own decisions hide its mistakes: cloud answers are harmful labels, and only a 5% operator audit, pooled across regions, keeps a clear advantage over the cascade (142.4). Learning to route under one-sided feedback, not model synchronization, is the bottleneck of federated decision-making in inspection fleets.

Data and code availability

The code, the cached tool answers and the result files are in the project repository, branch vision-fl (to be made public on acceptance). ElecVQA-Bench and InsPLAD are public [16, 17]. The laya-vision weights are published under CC BY-NC-SA 4.0 [6].

Declaration of competing interest

The author declares no competing interests.

CRediT authorship contribution statement

Mahdi Khashan: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft, Writing – review & editing.

Declaration of generative AI use

During the preparation of this work the author used an AI assistant to write code, analyse results and draft text. The author reviewed and edited the content and takes full responsibility for the publication.

Acknowledgments

Experiments in this work were run and tracked with OpenResearch [21].

References

[1] R. Farahani, N. Hossein Motlagh, Z. Azimi, C. Timmerer, L. Carnevale, S. Tarkoma, S. Dustdar, From pixels to semantics: Edge AI for UAV-based critical infrastructure inspection, arXiv:2609.18448 (2026).

[2] R. Farahani, Z. Azimi, I. Murturi, A. Goknil, S. Sen, C. Timmerer, S. Dustdar, EVLM: Intent-driven edge vision language model for UAV-based power line inspection, in: 2026 IEEE International Conference on Edge Computing and Communications (EDGE), 2026. https://doi.org/10.1109/EDGE72783.2026.00014.

[3] Z. Azimi, R. Farahani, S. Dustdar, C. Timmerer, Cloud, edge, or split? Profiling onboard and split vision-language model deployment for drone AI, arXiv:2609.25415 (2026).

[4] A. Marafioti, et al., SmolVLM: Redefining small and efficient multimodal models, arXiv:2504.05299 (2025).

[5] Convai Innovations, Laya, software (2026). https://github.com/NandhaKishorM/laya.

[6] Laya Vision, software, commit 9e1e241 (2026). https://github.com/r33drichards/laya-vision; weights https://huggingface.co/thaitea/laya-vision.

[7] Sabanovic, Maliakel, Brandic, INAR-VL: Input-aware routing for edge-cloud vision-language inference, arXiv:2605.18853 (2026).

[8] H. Zhang, S. Cha, H. B. Beytur, K. S. Chan, G. de Veciana, H. Vikalo, Online learning for multi-layer hierarchical inference under partial and policy-dependent feedback, arXiv:2603.04247 (2026).

[9] W. Wei, T. Yang, H. Chen, Y. Zhao, F. Dernoncourt, R. A. Rossi, H. Eldardiry, Learning to route LLMs from bandit feedback: One policy, many trade-offs, arXiv:2510.07429 (2025).

[10] A. Agarwal, H. B. McMahan, Z. Xu, An empirical evaluation of federated contextual bandit algorithms, arXiv:2303.10218 (2023).

[11] B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), 2017.

[12] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, V. Chandra, Federated learning with non-IID data, arXiv:1806.00582 (2018).

[13] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, V. Smith, Federated optimization in heterogeneous networks, in: Proceedings of Machine Learning and Systems (MLSys), 2020.

[14] Askin, et al., Federate the router: Learning language model routers with sparse and decentralized evaluations, arXiv:2601.22318 (2026).

[15] Solat, Lee, Seif, Niyato, Poor, FedHLM, IEEE Internet of Things Journal (2025). https://doi.org/10.1109/jiot.2025.3618095.

[16] ElecVQA-Bench, dataset and code repository (2026). https://github.com/GhostDino/elecvqa-bench.

[17] A. L. B. Vieira e Silva, et al., InsPLAD: A dataset and benchmark for power line asset inspection in UAV images, International Journal of Remote Sensing (2023). arXiv:2311.01619.

[18] J. van der Hooft, S. Petrangeli, T. Wauters, R. Huysegems, P. R. Alface, T. Bostoen, F. De Turck, HTTP/2-based adaptive streaming of HEVC video over 4G/LTE networks, IEEE Communications Letters 20 (11) (2016) 2177–2180.

[19] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[20] E. J. Hu, et al., LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations (ICLR), 2022.

[21] alphaXiv, OpenResearch: The local-first harness and workspace for research agents (2026). https://github.com/alphaXiv/OpenResearch.